

Mavenir RAG Chatbot

By Chitransh Saxena

A Retrieval-Augmented Generation system for 3GPP standards, built for minimal to near-zero hallucination

Technical Report and Design Documentation

<https://github.com/CSroseX/Mavenir-Assignment>

Contents

1. Project Overview	2
2. Tech Stack	2
3. Architecture	3
3.1 Ingestion pipeline.....	3
3.2 Retrieval.....	3
3.3 Generation and verification.....	3
4. Steps Taken to Reduce Hallucinations	3
4.1 Verified retrieval over guessed retrieval	4
4.2 Two-stage retrieval with reranking.....	4
4.3 Independent claim verification	4
4.4 Bounded retry with honest flagging.....	4
4.5 Grounded, non-evasive prompting.....	4
4.6 What this deliberately does not claim	4
5. Limitations and How They Were Solved.....	4
6. Alignment with Mavenir's Direction	5
6.1 Why this architecture fits that direction	5
6.2 Why this is achievable within realistic time and resources.....	5
7. Future Implementations	5
8. How to Run the Project.....	6
8.1 Prerequisites.....	6
8.2 Start Qdrant.....	6
8.3 Install dependencies	6
8.4 Configure environment	6
8.5 Run ingestion (first time only).....	6
8.6 Launch the app.....	6
9. Closing Notes.....	7

1. Project Overview

This Assignment was given in the selection process for Graduate Engineer Trainee position assigned to me at 13th August, 2026.

Assignment requirement – “ Chatbot built using a Retrieval-Augmented Generation (RAG) architecture, with a focus on achieving minimal to near-zero hallucinations. The solution should use Telecom 3GPP standards documentation as the primary knowledge source.”

This assignment is a chatbot that answers questions over 3GPP telecom specifications using retrieval-augmented generation. The goal was never just "a chatbot that answers questions." Anyone can wire an LLM to a vector store and get plausible-sounding answers. The actual goal, and the harder engineering problem, was building a system that knows the difference between what it actually knows from the source documents and what it is tempted to guess, and refuses to guess.

Telecom specs are an unusually good and unusually punishing testbed for this. 3GPP documents like TS 23.501 (5G system architecture), TS 24.501 (5G NAS protocol), and TS 24.301 (4G/EPS NAS protocol) share huge amounts of vocabulary and procedural structure between the 4G and 5G worlds. A model that is even slightly loose with its grounding will happily blend a 4G timer with a 5G procedure, or describe an AMF behavior using EPS terminology, because the two domains sound similar enough to pattern-match against. That overlap was chosen deliberately as a stress test, not avoided.

Everything below documents what was built, why it was built that way, and what tradeoffs were made under real hardware and time constraints.

2. Tech Stack

The stack was chosen with two constraints in mind: it had to run reliably without heavyweight infrastructure, and every component had to be swappable without a rewrite, since embedding and generation providers both hit quota or hardware limits during development.

Layer	Choice	Why
Document parsing	Docling	Structure-aware PDF/HTML parsing; needed workarounds for unreliable heading levels and header/footer labeling, both handled in the tree-building step rather than patched inside Docling itself.
Clause tree	Custom regex over clause numbers	Docling's own heading-level field was not reliable enough for a legal-code-style numbering scheme, so the hierarchy is rebuilt independently from the clause numbers themselves.
Chunking	Hierarchical, ~300-500 tokens	Tables kept anchored and intact rather than split; every chunk carries full lineage metadata (spec, clause path, parent headings).
Vector store	Qdrant (Docker, local)	Self-hosted, fast, supports payload-based metadata filtering alongside vector search, which the dual-path retriever depends on.
Embeddings	FastEmbed, BAAI/bge-small-en-v1.5 (local)	Runs fully locally with no API quota risk; small enough to embed the full corpus without a GPU.

Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2 (local)	Second-stage reranker over a wider candidate set; catches semantically correct matches that plain vector similarity ranks too low, without adding another API dependency.
Generation LLM	Groq-hosted Qwen model (OpenAI-compatible API)	Chosen for speed and availability after local hardware proved too constrained for a strong enough model; the client code is provider-agnostic so it can be pointed at any OpenAI-compatible endpoint, including a local one.
Orchestration	LangGraph	Explicit generate to verify_claims to retry graph, scoped only to generation, not ingestion. Gives visible, inspectable state at every step instead of one opaque generation call.
Frontend	Streamlit	Fast to build, good enough for a working demo with a chat view and an evaluation dashboard.

3. Architecture

3.1 Ingestion pipeline

Parse (Docling) -> rebuild the clause tree with regex -> hierarchical chunking (300-500 tokens, tables anchored) -> embed (FastEmbed) -> index (Qdrant). This is a one-time offline pipeline over the fixed corpus of three specs and does not run at query time.

3.2 Retrieval

Retrieval is dual-path, decided by whether the query names a spec or clause explicitly:

- Named spec or clause (e.g. "TS 24.501 clause 5.5.1") — detected with regex, then filtered directly in Qdrant using metadata, so the search is scoped to exactly the right document and clause family from the start.
- No spec named — semantic search runs across all three specs at once. If the same clause number exists in more than one spec, both are returned rather than the system silently guessing which one the user meant.

On top of this, retrieval is two-stage: a wider candidate set (15-20 chunks) is pulled from Qdrant first, then reranked with a local cross-encoder against the original query, and only the top handful are passed to the LLM. A single-stage vector search alone was tested first and consistently returned too few genuinely relevant chunks for multi-part or adversarial questions; see Section 5 for how this was diagnosed.

3.3 Generation and verification

Generation is a small LangGraph state machine, not a single prompt call:

- generate : answers the question using only the retrieved chunks, in plain expert prose, with no reference to retrieval mechanics.
- verify_claims : an independent LLM call checks whether the generated answer is actually entailed by the retrieved source text, not just plausible-sounding. This step treats the first answer as a hypothesis to be checked, not a final output.
- retry : if verification fails, the system retries generation with feedback on what was not supported, bounded to two or three attempts. If it still cannot produce a fully supported answer, the answer is flagged rather than shipped silently.

4. Steps Taken to Reduce Hallucinations

Hallucination reduction was treated as a pipeline-wide concern, not a single prompt trick. Five separate mechanisms work together:

4.1 Verified retrieval over guessed retrieval

The dual-path retriever means the system never has to guess which document a clause reference belongs to when the user names one explicitly. When no spec is named, ambiguity across specs is surfaced rather than resolved silently, which matters directly for 3GPP corpora since clause numbers are reused across specs with different meanings.

4.2 Two-stage retrieval with reranking

Early testing showed that pure vector similarity with a small top-k was the single biggest source of weak or unsupported answers, not the LLM itself. Widening the candidate pool and reranking with a cross-encoder measurably improved which chunks reached the model, which improved everything downstream.

4.3 Independent claim verification

This is the core safeguard. `verify_claims` does not check whether a citation exists, it checks whether the specific claim in the answer is actually entailed by the retrieved chunk content. An answer that sounds confident and complete but drifts from the source gets caught here before it ever reaches the user.

4.4 Bounded retry with honest flagging

If verification fails, the system gets a second and third attempt with explicit feedback about which claims were not supported. If it still cannot produce a grounded answer after the retry budget, the system says so in the UI rather than quietly returning its best guess. Hiding an unverifiable answer would defeat the purpose of having a verifier at all.

4.5 Grounded, non-evasive prompting

The generation prompt explicitly instructs the model to speak as a subject matter expert and to say plainly when the specifications do not define something, without ever exposing retrieval internals like chunk numbers to the end user. An early version of this prompt still leaked phrasing like "based on the provided source chunks," which was caught in testing and tightened with explicit banned phrases and a stronger persona instruction, plus a backstop text filter as a second line of defense.

4.6 What this deliberately does not claim

Zero hallucination in an absolute sense is not a claim this system makes, and no RAG system honestly can. What it does claim is that every answer either passes an independent grounding check or is explicitly flagged as unverified, so the failure mode is visible instead of hidden. That distinction, a system that knows when it does not know, is the actual deliverable.

5. Limitations and How They Were Solved

Several real problems came up during build and testing. Each one is listed here honestly along with the fix, since this is more informative than presenting a clean, frictionless story.

Limitation	How it was solved
Docling produced unreliable heading levels and mislabeled headers/footers on the source PDFs.	The clause hierarchy was rebuilt independently using regex over the clause numbering itself, instead of trusting Docling's structural field. This decoupled chunk structure from a parser quirk that was out of scope to fix upstream.
An early content filter deleted 1,383 real content blocks while trying to strip boilerplate.	Caught during a verification pass on the ingested corpus, root-caused to an overly aggressive filter rule, and fixed before indexing. Final corpus was re-verified at 4,860 chunks with zero boilerplate pollution.
Every chunk in the vector database was mislabeled with the same <code>spec_id</code> due to a missing parameter	Found while validating a cross-spec clause query that looked suspicious. Fixed at the source (the ingestion call), verified against the raw chunk

in the ingestion call, so document filtering was silently broken.	content itself (not just the fix script's own output), and confirmed independently against the live Qdrant database after a full, clean re-index.
A reasoning-capable LLM's internal "thinking" tokens sometimes ate the entire token budget before it could finish a structured JSON verification response, causing a hard crash.	Reasoning was explicitly disabled for this call at the API level rather than just hidden from the UI, which removed the root cause instead of the symptom, plus added graceful error handling so a verification failure degrades to a flagged answer instead of crashing the app.
Single-stage vector retrieval with a small top-k returned too few relevant chunks for adversarial or multi-part questions.	Moved to two-stage retrieval: a wider candidate pool from Qdrant, reranked locally with a cross-encoder, before the final top chunks are sent to the LLM.
Local hardware (16 GB RAM, integrated GPU with no usable VRAM for inference) could not run a generation model strong enough for reliable RAG.	Moved generation to a hosted, OpenAI-compatible endpoint while keeping the client code provider-agnostic, so the same pipeline can run against a local model on stronger hardware without a rewrite.

6. Alignment with Mavenir's Direction

Mavenir positions itself as an end-to-end, cloud-native network software provider, with a heavy emphasis on Open RAN, standards compliance, and software that runs on any cloud rather than proprietary hardware. This project was approached in that same spirit, applied to an AI use case rather than a RAN one.

6.1 Why this architecture fits that direction

- Standards-first: the entire system is built around correctly interpreting 3GPP specifications, the same documents that govern Mavenir's core, RAN, and edge products. A verified, grounded Q&A layer over these specs is directly useful for engineering teams who currently do this lookup manually.
- Cloud-native and portable: every component (Qdrant, the embedding model, the reranker) runs in standard containers or as local Python processes with no vendor lock-in. The generation layer is provider-agnostic by design, which matters given Mavenir's stated preference for infrastructure that runs on any cloud.
- Runs on modest infrastructure: this entire system, including a full 4,860-chunk corpus across three specifications, was built and demonstrated on a consumer laptop with 16 GB of RAM and no dedicated GPU. That is a meaningful signal: a small, focused team does not need a large GPU cluster to stand up a working, verifiably grounded RAG system over dense technical documentation. It scales up cleanly from there rather than requiring heavy infrastructure just to get started.

6.2 Why this is achievable within realistic time and resources

The build shows a clear, repeatable pattern: parse once, verify the ingested data independently before trusting it, retrieve with both keyword-aware and semantic paths, and never let a generated answer reach the user without a second, independent check. None of these steps require exotic infrastructure or a large model. The main cost driver is engineering discipline around verification, not compute. That is a good sign for scaling this approach to Mavenir's broader specification set (O-RAN, additional 3GPP releases, internal engineering documentation) without a proportional jump in infrastructure spend.

7. Future Implementations

- Expand the corpus beyond the three current specs to the full 3GPP Release 17/18 set relevant to Mavenir's product lines, plus O-RAN Alliance specifications.
- Add a formal evaluation harness that runs a fixed, labeled question set (normal, adversarial 4G/5G conflation, and out-of-scope refusal cases) on every change to retrieval or generation, so hallucination-relevant metrics are tracked over time rather than spot-checked.

- Add conversational memory for genuine multi-turn follow-up questions, kept separate from the grounding mechanism so it improves usability without weakening verification.
- Explore a larger or fine-tuned local model for generation once hardware allows, to reduce dependence on a hosted API and remove the remaining quota risk entirely.
- Add source-diffing for a specific, well-known 3GPP failure mode: automatically flagging when a question's answer differs meaningfully between spec releases (e.g. Release 15 vs Release 17), which is a common real-world confusion point for engineers.
- Add a feedback loop where flagged or user-corrected answers are logged and reviewed, to identify systematic gaps in the corpus or the chunking strategy over time.
- Harden the reranker and retrieval further with a larger cross-encoder or a fine-tuned domain-specific reranker trained on 3GPP terminology, if evaluation shows current reranking as a bottleneck.

8. How to Run the Project

8.1 Prerequisites

- Python 3.10 or later
- Docker (for running Qdrant locally)
- An API key for the generation provider (or a local OpenAI-compatible endpoint, e.g. Ollama, if running generation locally instead)
- Roughly 2 GB of free disk space for the corpus, embeddings index, and local models (embedding model and reranker)

8.2 Start Qdrant

Qdrant runs locally in Docker and does not require an account or cloud setup:

```
docker run -p 6333:6333 -p 6334:6334 \ -v $(pwd)/qdrant_storage:/qdrant/storage \
qdrant/qdrant
```

This exposes the Qdrant REST/gRPC API on localhost:6333. The -v flag persists the vector index to disk so it survives a container restart.

8.3 Install dependencies

```
python -m venv venv venv\Scripts\activate (Windows) or source venv/bin/activate
(macOS/Linux) pip install -r requirements.txt
```

8.4 Configure environment

Set the generation provider's API key and endpoint in a .env file in the project root:

```
GROQ_API_KEY=your_key_here MODEL_NAME=your_model_name_here
```

8.5 Run ingestion (first time only)

This parses the source specs, builds the clause tree, chunks the content, generates embeddings, and indexes everything into Qdrant. It only needs to be run once, or again if the source documents or chunking logic change.

```
python src/ingestion/indexer.py
```

8.6 Launch the app

```
streamlit run app.py
```

This opens the chat interface in the browser, where questions can be asked against the indexed corpus. Each answer shows its grounding status and the exact spec and clause sources it was built from.

9. Closing Notes

The through-line across this whole build is the same idea applied at every layer: do not trust an intermediate result just because it looks reasonable, verify it independently, and if it cannot be verified, say so rather than hide it. That applied to the ingested chunks, to the retrieval results, and most importantly to the final generated answer. For a domain like 3GPP specifications, where a confidently wrong answer is arguably worse than no answer at all, that principle is the actual product.

Created by: Chitransh Saxena

Date: 17th August, 2026